

Wllama - LLM-Inferenz im Webbrowser per WebAssembly

53. Sitzung der offenen Community of Practice #CoP_KIPerWeb

f-bb Forschungsinstitut Betriebliche Bildung

Agenda

1. Update (allgemein & persönlich)

2. Input: „Wllama - LLM-Inferenz im Webbrowser per WebAssembly“

3. Diskussion

Update

allgemein & persönlich

Forschungsinstitut
Betriebliche Bildung

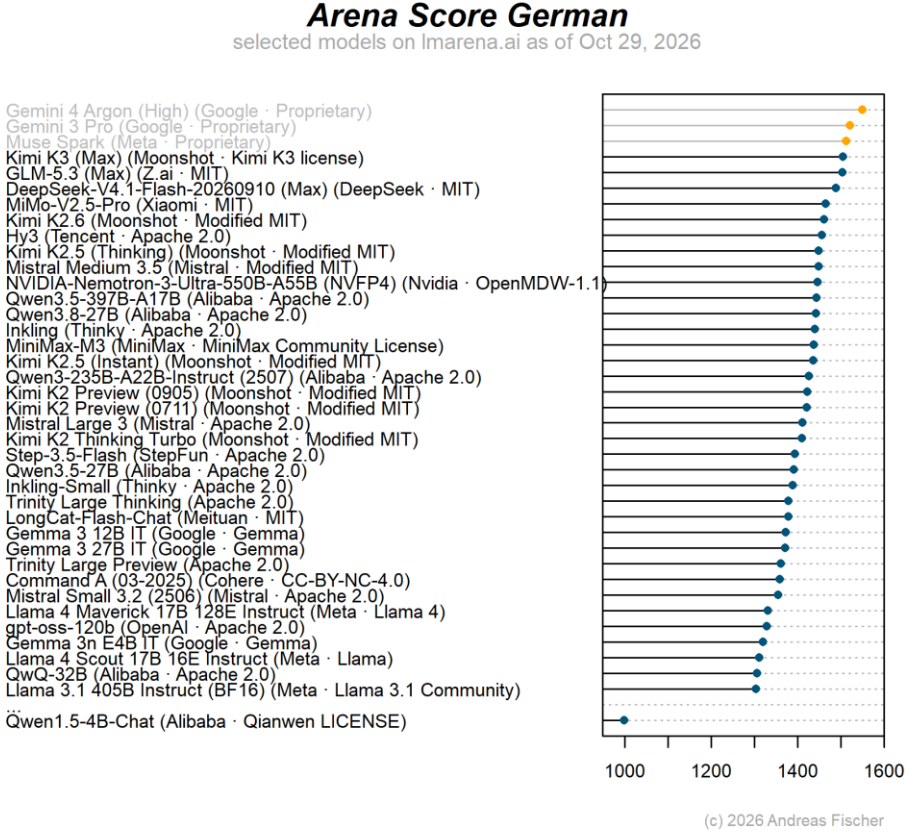
Update (allgemein)

Hintergrund:

- Rechts abgebildet ist auszugweise das aktuelle LMArena-Leaderboard für die Kategorie „German“ mit Blick auf die besten open-weights-Modelle
- je 1 pro Modellfamilie, zzgl. kleine Modelle (<30B) sowie die Top-3 proprietären Modelle

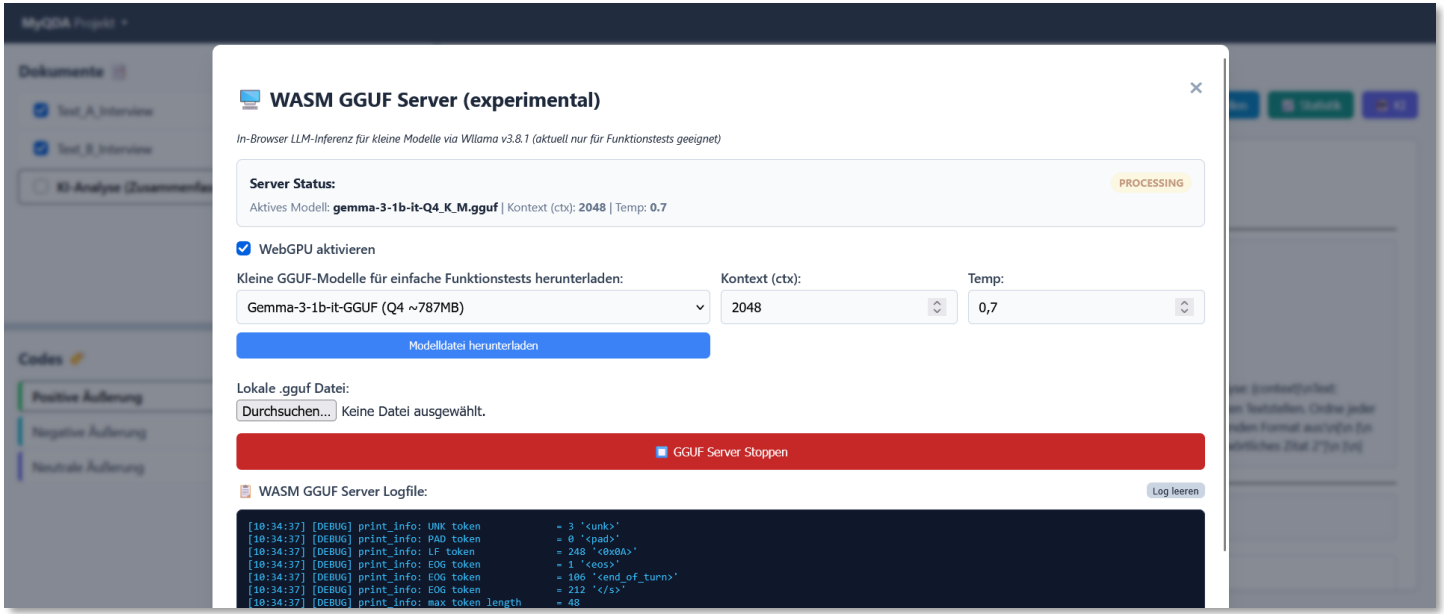
Bemerkenswert:

- Spitzenposition hat damit nun Gemini 4 Argon (nun vor Gemini 3 Pro)
- Bestes Open-Weights-Modell ist nach wie vor Kimi k3 max (2.8 Trillion Parameter, 104B aktiv)
- Effizientestes Open-Weights Modell bleibt Qwen3.8-27b, wobei gemma-4 nach wie vor nicht ausgewiesen ist.



Update (persönlich) – MyQDA-Updates

- Seit MyQDA-Update v1.22 (wllama) und v1.23 (WebGPU optional) mit integriertem GGUF-Model-Server:



Quelle: https://andreasfischer1985.github.io/MyQDA/src/MyQDA.html (Stand: 29.11.26)

Input

Wllama - LLM-Inferenz im Webbrowser per WebAssembly

Forschungsinstitut Betriebliche Bildung

WebAssembly

28 Sprachen

Artikel | Diskussion | Lesen | Bearbeiten | Quelltext bearbeiten | Versionsgeschichte

WebAssembly (**Wasm**[2]) ist ein offener Standard, der vom W3C festgelegt wurde. Er definiert einen Bytecode zur Ausführung von Programmen innerhalb von Webbrowsern, kann aber auch außerhalb von diesen genutzt werden. Ziel der Entwicklung war es, leistungsstärkere Webanwendungen als bisher zu ermöglichen, sowohl was die Ladezeiten als auch die Ausführung betrifft. Das Projekt wird von allen großen Entwicklern von Browser-Engines, also Mozilla, Microsoft, Google und Apple, unterstützt.[3][4]

Seit März 2017 wird die Version 1 standardmäßig mit Chrome, Firefox, Edge und Webkit ausgeliefert.[5][6] Der Standard wurde am 5. Dezember 2019 offiziell festgelegt.[7][8] Weitere Funktionen wurden einzeln standardisiert und sind in Entwicklung oder in einigen Implementierungen bereits enthalten.[9] Version 2 der Spezifikation lag seit April 2022 als Entwurf vor.[10] Eine Version 3 wurde im September 2025 vorgestellt.[11]

WebAssembly

WA

Basisdaten

Erscheinungsjahr:	2017
Designer:	World Wide Web Consortium
Entwickler:	Bytecode Alliance
Aktuelle Version	2.0[1] (1. Juni 2022)
Beeinflusst von:	asm.js, Google Native Client
Lizenz:	Apache-Lizenz

webassembly.org

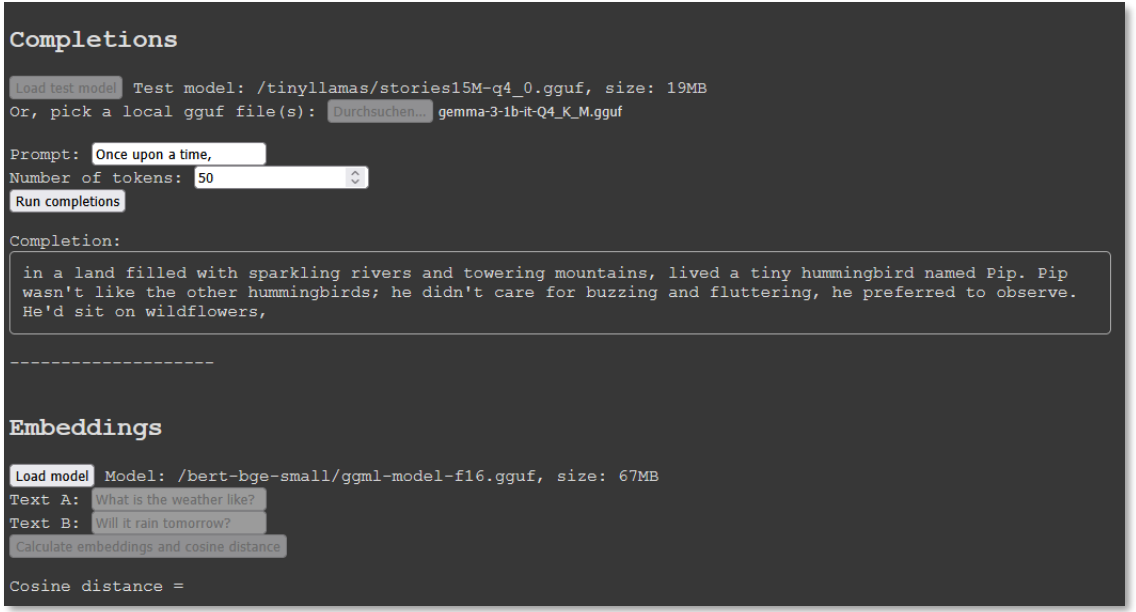
Quelle: https://de.wikipedia.org/wiki/WebAssembly

WASM-based In-Browser LLM-Inference

- **Wllama:** Fokus auf GGUF-Format (z.B. LFM3-VL-2B.Q4_K_M.gguf; LFM2.5-VL-450M-Q4_0.gguf**)**; WebGPU optional;
  - Demo: https://huggingface.co/spaces/ngxson/wllama
- **WebLLM:** Fokus auf MLC-Format (z.B. Qwen3-1.7B-q4f16_1-MLC); WebGPU obligat;
  - Demo: https://chat.webllm.ai/ ; https://huggingface.co/spaces/mlc-ai/webllm-playground
- **LiteRT.js:** Fokus auf Litert-Format (z.B. gemma-4-E2B-it-gpu.litertlm); WebGPU optional
  - Demo: https://huggingface.co/spaces/tylermullen/Gemma4
- **Transformers.js:** Fokus auf ONNX-Format (SmolLM-360M-Instruct); WebGPU optional)
  - Demos: https://github.com/huggingface/transformers.js-examples
- **Sonstiges:** Custom WASM-Builds (z.B. llama.cpp vis Emscripten), LiteRT.js, Tensorflow.js, MediaPipe, …

Wllama

- 🔌 OpenAI-compatible API (fully-typed built-in)
- 🚀 WebGPU support
- 🔥 Multimodal support (image and audio file input)
- 🔥 Tool calling support
- Can run inference directly on browser (using WebAssembly SIMD), no backend or GPU is needed!
- No runtime dependency (see package.json)
- Ability to split the model into smaller files and load them in parallel (same as split and cat)
- Auto switch between single-thread and multi-thread build based on browser support
- Inference is done inside a worker, does not block UI render
- Pre-built npm package @wllama/wllama



Quelle: https://github.ngxson.com/wllama/examples/basic/
https://github.com/ngxson/wllama

Wllama v3 Demo

- **Benutzeroberfläche bereitstellen:**
  Eine einfache HTML-Struktur stellt ein Dateiauswahlfeld für das .gguf-Modell, ein Textfeld für den Prompt, einen Sende-Button sowie eine Textarea für die Ausgabe bereit.
- **Umgehung von CORS- und Dateisystem-Restriktionen:**
  Das Hauptmodul von wllama wird dynamisch per fetch heruntergeladen, Pfade werden angepasst und das Skript wird als lokale Blob-URL ausgeführt, um den Betrieb direkt aus dem Dateisystem zu ermöglichen.
- **Modell-Initialisierung & Chat-Templates:**
  Nach Auswahl der lokalen GGUF-Datei (<2GB, sonst: *llama-gguf-split*) wird wllama initialisiert und das Modell im Arbeitsspeicher geladen, wobei aktivierte Jinja-Templates (*jinja: true*) die korrekte Formatierung für Instruktionsmodelle sicherstellen.
  - mit loadModel-Parameter *n_gpu_layers: 0 teils* mehr t/s 🤡
- **Asynchrone Inferenz & Live-Streaming:**
  Über die OpenAI-kompatible Schnittstelle (createChatCompletion) wird die Anfrage generiert und empfangene Textfragmente (delta) werden Token für Token live in die Textarea geschrieben.

Quelle V3: https://andreasfischer1985.github.io/code-snippets/html/wllamaDemo_v3.8.1.html

```html
<!DOCTYPE html><html lang="de"><head> <meta charset="UTF-8"><title>wllama Demo</title> </head><body>
    <input type="file" id="modelFile" accept=".gguf"><br><br>
    <input type="text" id="prompt" value="Schreibe einen kurzen Witz." style="width:300px;">
    <button onclick="run()" id="sendBtn" disabled>Senden</button><br><br>
    <textarea id="output" rows="10" cols="50" readonly>Lade wllama Runtime...</textarea>
    <script>
        let WllamaClass = null;
        (async function() {
            try {
                const version = '3.8.1';
                const moduleUrl =
            `https://cdn.jsdelivr.net/npm/@wllama/wllama@${version}/esm/index.js`;
                const res = await fetch(moduleUrl);
                let code = await res.text();
                code = code.replaceAll('./single-thread/',
            `https://cdn.jsdelivr.net/npm/@wllama/wllama@${version}/esm/single-thread/`);
                code = code.replaceAll('./multi-thread/',
            `https://cdn.jsdelivr.net/npm/@wllama/wllama@${version}/esm/multi-thread/`);
                const blob = new Blob([code], { type: 'application/javascript' });
                const blobUrl = URL.createObjectURL(blob);
                const wllamaModule = await import(blobUrl);
                WllamaClass = wllamaModule.Wllama;
                document.getElementById('output').value = "Bereit! Bitte GGUF-Modell auswählen.";
                document.getElementById('sendBtn').disabled = false;
            } catch (err) {
                console.error('Ladefehler:', err);
                document.getElementById('output').value = "Fehler beim Initialisieren: " + err.message;
            }
        })();
        window.run = async () => {
            if (!WllamaClass) return;
            const fileInput = document.getElementById('modelFile');
            const promptText = document.getElementById('prompt').value;
            const output = document.getElementById('output');
            if (fileInput.files.length === 0) return;
            output.value = "Lade Modell in den Speicher (kann einen Moment dauern)...";
            const wllama = new WllamaClass({
                'default': 'https://cdn.jsdelivr.net/npm/@wllama/wllama@3.8.1/esm/wasm/wllama.wasm'
            });
            await wllama.loadModel([fileInput.files[0]], { jinja: true });
            output.value = "";
            const chatStream = await wllama.createChatCompletion({
                messages: [{ role: 'user', content: promptText }], n_predict: 256, stream: true
            });
            for await (const chunk of chatStream) {
                const delta = chunk.choices[0]?.delta?.content || '';
                output.value += delta;
            }
            await wllama.exit();
        };
    </script></body></html>
```

Exkurs: Wllama v2 Demo

Angefangen hatte ich mich v2:

- Einfacher zu laden (ohne fetch-blob-workaround), aber ohne jinja-templates
- By default scheinbar schneller als v3 😳
  - v3 nutzt by default WebGPU, was bei Rechnern ohne dedizierte GPU aber zu Performance-Einbrüchen führen kann
  - *n_gpu_layers: 0* führte in v3 auf meinem Testrechner zu
    - mehr Tokens pro Sekunde (t/s)
    - höherer Time to First Token (TTFT)

Quellcode v2: https://andreasfischer1985.github.io/code-snippets/html/wllamaDemo_v2.3.3.html

```html
<!DOCTYPE html><html lang="de"><head><meta charset="UTF-8"><title>wllama v2.3.3 Demo</title></head><body>
    <input type="file" id="modelFile" accept=".gguf"><br><br>
    <input type="text" id="prompt" value="Schreibe einen kurzen Witz." style="width:300px;">
    <button id="sendBtn" disabled>Senden</button><br><br>
    <textarea id="output" rows="10" cols="50" readonly>Lade wllama Runtime...</textarea>
    <script type="module">
        import { Wllama } from 'https://cdn.jsdelivr.net/npm/@wllama/wllama@2.3.3/esm/index.js';
        const CONFIG_PATHS = {
            'single-thread/wllama.wasm': 'https://cdn.jsdelivr.net/npm/@wllama/wllama@2.3.3/esm/single-thread/wllama.wasm',
            'multi-thread/wllama.wasm': 'https://cdn.jsdelivr.net/npm/@wllama/wllama@2.3.3/esm/multi-thread/wllama.wasm'
        };
        let wllama = null;
        const output = document.getElementById('output');
        const sendBtn = document.getElementById('sendBtn');
        const fileInput = document.getElementById('modelFile');
        (async function() {
            try {
                wllama = new Wllama(CONFIG_PATHS);
                output.value = "Bereit! Bitte GGUF-Modell auswählen.";
                fileInput.disabled = false;
            } catch (err) {
                console.error('Ladefehler:', err);
                output.value = "Fehler beim Initialisieren: " + err.message;
            }
        })();
        fileInput.addEventListener('change', async (e) => {
            if (e.target.files.length === 0) return;
            output.value = "Lade Modell in den Speicher (kann einen Moment dauern)...";
            sendBtn.disabled = true;
            try {
                await wllama.loadModel([e.target.files[0]], { n_ctx: 2048 });
                output.value = "Modell erfolgreich geladen! Du kannst jetzt Senden.";
                sendBtn.disabled = false;
            } catch (err) {
                console.error('Modell-Ladefehler:', err);
                output.value = "Fehler beim Laden des Modells: " + err.message;
            }
        });
        sendBtn.addEventListener('click', async () => {
            if (!wllama) return;
            const promptText = document.getElementById('prompt').value;
            output.value = "";
            const formattedPrompt = `<|im_start|>user\n${promptText}<|im_endto|\n<|im_start|>assistant\n`;
            try {await wllama.createCompletion(formattedPrompt, {
                    nPredict: 256, sampling: { temp: 0.7, top_p: 0.9, top_k: 40 },
                    onNewToken: (token, piece) => {
                        let textChunk = '';
                        if (typeof piece === 'string') {textChunk = piece;} else if (piece instanceof Uint8Array) {
                            textChunk = new TextDecoder().decode(piece);} else if (token) {
                            try {textChunk = wllama.detokenize([token]);} catch (e) {textChunk = String(piece || '');}
                        }
                        output.value += textChunk;
                    }
                });
            } catch (err) {
                console.error('Inferenzfehler:', err);
                output.value += "\n[Fehler bei der Inferenz: " + err.message + "]";
            }
        });
    </script></body></html>
```

Diskussion und Austausch

Fragen, Anmerkungen, Erfahrungen?

Kontakt

Forschungsinstitut Betriebliche Bildung (f-bb)
Rollnerstraße 14, 90408 Nürnberg

Web: www.f-bb.de

Ansprechperson

Dr. Andreas fischer

Telefon: 0911 277 79-0
Mail: andreas.fischer@f-bb.de

f-bb Forschungsinstitut Betriebliche Bildung